

Egison Core

Demand-Directed Typing の形式仕様

目次

1	目的と読み方	2
2	構文と判断	3
3	一回の checking cut	6
4	式の synthesis 規則	7
5	Pattern と matcher の規則	9
6	機械化された性質と接続	12

1 目的と読み方

1.1 公開判断と三つの状態

本稿は, Egison core の source program が型付け可能であることを定める. 公開する source typing は `DDTyping` 一つだけであり, その内部を型の合成 (synthesis) と検査 (checking) に分ける. signature $\hat{\Sigma}$ は全判断で固定し, 次のように省略する.

$$\begin{aligned} q; S; \Omega; \Gamma \vdash e \Rightarrow \tau_{\text{raw}} \dashv q'; S'; \Omega' & \quad \text{synthesis,} \\ q; S; \Omega; \Gamma \vdash e \Leftarrow \rho \dashv q'; S'; \Omega' & \quad \text{checking.} \end{aligned}$$

ここで $q = (q_\kappa, q_\tau)$ は二 sort の fresh supply, S は prevailing paired substitution, Ω は capability variable の生成由来を記録する origin ledger である. q_κ と q_τ は, それぞれ次に生成する capability variable χ_{q_κ} と target variable α_{q_τ} の番号を持つ. どの規則も子を左から右へ調べ, 出力 $q'; S'; \Omega'$ を次の子へ渡す.

closed wrapper は canonical initial supply q_0 , 恒等置換, 空 ledger から開始し, 最後にだけ raw result を解決する.

$$\text{DDTyping}(\hat{\Sigma}, \Gamma, e, \tau) \iff \exists \tau_{\text{raw}}, q', S', \Omega', \mathcal{O}. q_0; \text{id}; \emptyset; \Gamma \vdash e \Rightarrow \tau_{\text{raw}} \dashv q'; S'; \Omega' \wedge \text{TerminalAudit}(S', \mathcal{O}) \wedge \tau = S' \tau_{\text{raw}}.$$

Lean では raw derivation と, それに構造を一致させた intrinsic Origin certificate を別 family として持つ. $\mathcal{O}$ はその Origin proof であり, terminal audit は同じ proof を再帰的に辿る. 上の記法は三者を一つに合わせた公開判断を表す. 追加 facts を持つ node は `let`, `matcher`, `pattern constructor` の三種である.

動的安全性の証明では, これら三つの推論状態を消去した内部 certificate `RuntimeTyping` を使う. これは source acceptance を定める第二の型システムではなく, runtime value typing と preservation の帰納法を状態履歴から切り離すための補助判断である. source acceptance と動的安全性に必要な構成要素の依存方向は

$$\text{infer success} \implies \text{DDTyping} \implies \text{RuntimeTyping} \implies \text{RuntimeSafety}$$

である. 最初の矢印は成功した traversal の相互 DD 再構成と terminal validator として機械化済みである. 二番目は closed signature ・ 空 context に特殊化した Origin derivation, 終端 substitution に固定した相互 state erasure, terminal audit の合成である. 最後は内部 certificate に対する `CoreSafety` の性質を模式的に表す. `DDTyping` を入口として後二つを束ねた source-facing な公開安全性定理は今後の接続である.

1.2 Origin ledger が表すもの

$\Omega(\chi)$ は次の三値を取る. ledger に明記されない variable は rigid として扱う.

rigid	signature または入力 context に由来し, solve で動かさない;
renameOnly	外へ流れる producer であり, 非 structural variable への rename だけを許す;
structuralFlexible	constructor 内部または consumer demand の局所 variable であり, export 前の structural solve を許す.

expression scheme と pattern-function dual scheme の capability binder は, instance 時点から `renameOnly` である. constructor / primitive instance と fresh consumer は `structuralFlexible` として生成する. constructor export は公開 payload に残る像の structural leaf だけを選択的に freeze する. matcher finalization は, 最終 capability に現れる全 DD-owned explicit ledger key の structural leaf を freeze するため, matcher 開始前に生成された `fixMatcher` placeholder の owned leaf も対象になる. ordinary equality と one-way solve は, いずれもその cut の ledger に対して admissible でなければならない.

1.3 中心となる考え：producer と slot demand

$\text{Matcher } \kappa \tau$ は matcher を生成する producer type, $\text{MatcherSlot } \kappa \tau$ は matcher を消費する位置の expected type である。checking は常に

$\text{synthesize } e \longrightarrow \text{その出力 cut で一度だけ expected type と align する}$

という手順を取る。非恒等 coercion を選べるのは、その cut で resolved expected head が MatcherSlot と判明している場合だけである。したがって coercion の場所は `matchAll` など特定の構文に固定されない。例えば

$$\text{use} : \text{MatcherSlot } \kappa \tau \rightarrow \rho, \quad m : \text{Matcher } \kappa \tau$$

の下で $\text{use } m$ を型付けすると、関数適用が domain を先に確定し、引数 m の checking cut で producer と slot が対応付けられる。`matchAll` の matcher 引数と matcher clause の next matcher も同じ checking を使う。「一度だけ」は導出木全体ではなく、一つの checking cut ごとに一度という意味である。

resolved source と expected の組は次の表で分類する。

resolved source	resolved expected	alignment
$\text{Matcher } \kappa_p \tau_p$	$\text{MatcherSlot } \kappa_c \tau_c$	matcher-to-slot
product of matchers	$\text{MatcherSlot } \kappa_c \tau_c$	product lift; matcher-to-slot
product of slots	$\text{MatcherSlot } \kappa_c \tau_c$	slot-tuple lift
$\text{MatcherSlot } \kappa_p \tau_p$	$\text{MatcherSlot } \kappa_c \tau_c$	slot-to-slot equality
その他	その他	ordinary equality

expected が型変数のままなら ordinary equality だけを行う。coercion のために未解決変数を slot へ推測せず, ordinary equality の失敗後に別 branch を試す rollback も行わない。

1.4 章の案内

第 2 章は型・source form・三状態と judgment の記号を導入する。第 3 章は一回の checking cut と ledger-aware alignment, 第 4 章は通常の式, 第 5 章は pattern と matcher literal の規則である。第 6 章は solved-form 保存, terminal audit, state erasure, 安全性への接続をまとめる。Lean module と回帰の詳細は `../docs/details.md` に譲る。

2 構文と判断

2.1 型とスキーム

capability κ と target type τ は別 sort である。matcher と slot は同じ二つの index を持つが、前者は producer, 後者は consumer expectation を表す。

$$\begin{array}{ll}
\kappa ::= \text{Any} \mid \chi \mid \hat{s} \mid K \bar{\kappa} \mid (\kappa_1, \dots, \kappa_n) & \kappa : \text{Cap}, \\
\tau ::= \alpha \mid \hat{a} \mid \mathbf{1} \mid \text{Int} \mid \text{Bool} \mid K \bar{\tau} \mid (\tau_1, \dots, \tau_n) \mid \tau_1 \rightarrow \tau_2 & \tau : \text{Type}, \\
& \mid \text{Matcher } \kappa \tau \mid \text{MatcherSlot } \kappa \tau & \tau : \text{Type}, \\
\sigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). \tau & \text{expression scheme,} \\
\eta ::= \kappa \triangleright \tau & \text{pattern dual,} \\
\varsigma ::= \forall(\bar{\chi} : \text{Cap}). \forall(\bar{\alpha} : \text{Type}). (\bar{\eta} \Rightarrow \eta) & \text{pattern-function scheme.}
\end{array}$$

χ, α は flexible metavariable, $\hat{s}, \hat{a}$ は rigid skolem である。Any は one-way capability matching の consumer 側でだけ wildcard として働き, 対称 unification では通常の rigid constructor である。mono τ は量化 binder を持たない

scheme を表す.

上の $\forall\bar{x}.\forall\bar{\alpha}$ は紙面上の表記である. Lean の canonical Scheme は binder 数と PolyCap/PolyTy payload を持ち, bound occurrence を Fin index, 自由な solver metavariable を CapVar/TyVar として別 datatype で表す. 通常型 κ, τ と unifier には bound constructor を追加しない. 従って metavariable substitution は scheme の自由 occurrence だけを書き換え, bound index を捕捉できない. generalization は選んだ metavariable を直ちに finite index へ close し, instance は有限 opening を通してのみ通常型へ戻す. payload は構成時から alpha-normal form である.

2.2 ソース構文

$$\begin{aligned}
e &::= x \mid \lambda x.e \mid e_1 e_2 \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{fix } f x.e \mid n \\
&\quad \mid (e_1, \dots, e_n) \mid C \bar{e} \mid op(\bar{e}) \mid \text{something} \\
&\quad \mid \text{matcher } cls \\
&\quad \mid \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e, \\
p &::= \$x \mid _ \mid \#e \mid \tilde{x} \mid c \bar{p} \mid p_1 \& p_2 \mid p_1 \mid p_2 \mid f \bar{p} \mid (\bar{p}), \\
pp &::= \$ \mid _ \mid \#\$x \mid c \bar{pp} \mid (\bar{pp}), \\
dp &::= \$x \mid _ \mid C \bar{dp} \mid (\bar{dp}), \\
cl &::= pp \text{ as } M \text{ with } [dp_j \rightarrow N_j]_{j=1}^r, \\
cls &::= [cl_i]_{i=1}^m.
\end{aligned}$$

c は pattern constructor, C は data constructor である. surface match は matchAll と head への elaboration であり, core primitive ではない. pattern-function declaration は source expression の typing より前に freeze され, canonical signature の一部になる.

2.3 シグネチャ・文脈・状態

$ \begin{aligned} \hat{\Sigma} &= (\hat{\Sigma}_D, \hat{\Sigma}_P, \hat{\Sigma}_F, \hat{\Sigma}_{prim}, Obs_{\hat{\Sigma}}) \\ \Gamma &::= \emptyset \mid \Gamma, x : \sigma \\ \Phi &::= \emptyset \mid \Phi, x : \eta \\ \Delta &::= \emptyset \mid \Delta, x : \tau \\ q &= (q_\kappa, q_\tau) \\ S &= (C, T) \end{aligned} $	freeze 済み canonical signature, expression scheme context, pattern-parameter context, monomorphic pattern bindings, next capability / target variable numbers, prevailing paired substitution, capability-origin ledger.
--	--

$\Omega : \chi \rightarrow \{\text{rigid}, \text{renameOnly}, \text{structuralFlexible}\}$

q_κ と q_τ は独立な自然数 counter であり、それぞれ次に未使用の capability metavariable と target metavariable の番号を保持する。以下では

$$\chi_{q_\kappa} : \text{Cap}, \quad \alpha_{q_\tau} : \text{Type}, \quad q + \langle i, j \rangle = (q_\kappa + i, q_\tau + j)$$

と書く。従って capability variable を一つ生成すれば supply は $q + \langle 1, 0 \rangle$, target variable なら $q + \langle 0, 1 \rangle$, 両方を一つずつ生成する pattern variable 規則では $q + \langle 1, 1 \rangle$ となる。番号空間は sort ごとに別なので、 χ_3 と α_3 は別の変数である。closed wrapper の初期 supply は, signature と context に既出の番号より各 counter が大きくなるように選ぶ。

S の作用は capability を先に, target substitution を後に適用する。

$$S\kappa = C\kappa, \quad S\tau = T(C\tau), \quad S\Gamma = \{x : S\sigma \mid x : \sigma \in \Gamma\}.$$

substitution delta は seq で時系列順に prevailing substitution へ合成する。後段の capability action は前段の target range 内部にも作用するため、二 component を独立に合成しない。

Ω は capability variable がどこで生成され、現在どの solve を許すかを記録する。未登録の key は rigid である。scheme / dual instance の binder image は `renameOnly`, constructor / primitive instance と fresh consumer は `structuralFlexible` になる。export は外へ残る像の structural leaf だけを `renameOnly` へ移す。matcher finalization は最終 capability に現れる全 DD-owned explicit ledger key を調べるため、matcher 開始前の owned placeholder も freeze の対象となる。

2.4 判断の一覧

$q; S; \Omega; \Gamma \vdash e \Rightarrow \tau \dashv q'; S'; \Omega'$	expression synthesis
$q; S; \Omega; \Gamma \vdash e \Leftarrow \rho \dashv q'; S'; \Omega'$	expression checking
$q; S; \Omega; \Gamma \vdash \bar{e} \Rightarrow \bar{\tau} \dashv q'; S'; \Omega'$	expression-list traversal
$q; S; \Omega; \Gamma; \Phi; \Delta \vdash p \Rightarrow \eta; \Delta' \dashv q'; S'; \Omega'$	user-pattern synthesis
$q; S; \Omega \vdash pp \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'$	primitive-pattern checking
$q; S; \Omega \vdash dp \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'$	data-pattern checking
$q; S; \Omega; \Gamma \vdash cl \Leftarrow \tau \Rightarrow \bar{\eta} \dashv q'; S'; \Omega'$	matcher-clause inference
$q; S; \Omega; \dots \vdash \bar{a} \text{ or } \bar{cl} \dashv q'; S'; \Omega'$	arm / clause-list traversal

list judgment は空列で q, S, Ω をそのまま返す。cons では head の出力を tail の入力にし、binding context も左から右へ延長する。lookup または規則中の部分関数が失敗した場合、その規則は適用できない。

2.5 具体化と一般化

Inst_q と InstDual_q は supply-indexed に量化 binder を fresh variable へ置き換え、次 supply も返す。expression context と pattern-function signature の量化 capability binder は capability variable へだけ instantiate され、その像を Ω に `renameOnly` として登録する。一方、data / pattern constructor と primitive の scheme binder は `structuralFlexible` として登録し、局所的な structural instance を許す。利用側の demand だけを根拠に producer capability を constructor や product へ変えることはできない。

Gen は signature と context に自由な variable を除いて量化する。

$$\text{Gen}(\widehat{\Sigma}, \Gamma, \tau) = \forall(\text{fcv}(\tau) \setminus (\text{fcv}(\widehat{\Sigma}) \cup \text{fcv}(\Gamma))). \forall(\text{ftv}(\tau) \setminus (\text{ftv}(\widehat{\Sigma}) \cup \text{ftv}(\Gamma))). \tau.$$

scheme binder は局所的であり、後段の substitution が導入した同番号の自由 variable を捕捉しない。let は value の synthesis 直後の $S\Gamma, S\tau$ を一般化する。さらに body の終端 substitution S' に対して

$$S'\sigma = \text{Gen}(\widehat{\Sigma}, S'\Gamma, S'\tau)$$

が成り立つことを Origin certificate に要求する。これは generalization 後の binder を後続 solve が遡及的に構造化しないための terminal stability 条件である。

3 一回の checking cut

3.1 二つの alignment 判断

coercion を伴わない equality alignment と checking cut 全体を, それぞれ

$$S; \Omega \vdash \tau \doteq \rho \dashv S', \quad S; \Omega \vdash \tau \preceq \rho \dashv S'$$

と書く. どちらも constraint を解く局所 delta を入力 S の後へ時系列順に合成する. ledger Ω 自体は solve では変化しないが, delta が Ω に対して admissible であることを要求する.

ExactCapMGU $_{\Omega}$ と ExactPairedMGU $_{\Omega}$ は, capability と paired type constraint の proof-carrying MGU に origin admissibility を加えたものである. MGU の soundness と因子化に加え, constraint 外で恒等, range が constraint 内, solved form が冪等であることを要求する. さらに rigid key は固定し, renameOnly key は非 structural variable へしか写さない. OneWayDelta $_{\Omega}$ は producer を固定したまま consumer capability と target を解く exact delta であり, 同じ admissibility 条件を満たす.

3.2 通常の等価整合

matcher 同士または slot 同士では capability を先に解き, その delta を target へ作用させてから target constraint を解く. それ以外の型は paired type 全体を一度に解く.

$$\frac{S\tau = \text{Matcher } \kappa_1 \tau_1 \quad S\rho = \text{Matcher } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_{\Omega}(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_{\Omega}(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-MATCHER}]$$

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_{\Omega}(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_{\Omega}(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))} \quad [\text{ALIGN-SLOT}]$$

$$\frac{\text{alignPairClass}(S\tau, S\rho) = \text{ordinary} \quad \text{ExactPairedMGU}_{\Omega}(S\tau, S\rho, D)}{S; \Omega \vdash \tau \doteq \rho \dashv \text{seq}(D, S)} \quad [\text{ALIGN-ORDINARY-TYPES}]$$

3.3 Demand classifier と coercion の選択

demandClass($S\tau, S\rho$) は resolved type だけから productMatcherLift, slotTupleLift, matcherToSlot, slotToSlot, ordinary のいずれかを定める. product of matchers, product of slots の順に調べるため, 空 product は前者が優先される.

$$\frac{\text{productMatcherDUALS?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{OneWayDelta}_\Omega((\bar{\eta}.\bar{\kappa}), (\bar{\eta}.\bar{\tau}), \kappa, v, D)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(D, S)}$$

[ALIGN-PRODUCT-MATCHER]

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{slotTupleLift} \quad \text{productSlotDUALS?}(S\tau) = \text{some } \bar{\eta} \quad S\rho = \text{MatcherSlot } \kappa v \quad \text{ExactCapMGU}_\Omega((\bar{\eta}.\bar{\kappa}), \kappa, C) \quad \text{ExactPairedMGU}_\Omega(C(\bar{\eta}.\bar{\tau}), Cv, T)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))}$$

[ALIGN-SLOT-TUPLE]

$$\frac{S\tau = \text{Matcher } \kappa_p \tau_p \quad S\rho = \text{MatcherSlot } \kappa_c \tau_c \quad \text{OneWayDelta}_\Omega(\kappa_p, \tau_p, \kappa_c, \tau_c, D)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(D, S)}$$

[ALIGN-MATCHER-SLOT]

$$\frac{S\tau = \text{MatcherSlot } \kappa_1 \tau_1 \quad S\rho = \text{MatcherSlot } \kappa_2 \tau_2 \quad \text{ExactCapMGU}_\Omega(\kappa_1, \kappa_2, C) \quad \text{ExactPairedMGU}_\Omega(C\tau_1, C\tau_2, T)}{S; \Omega \vdash \tau \preceq \rho \dashv \text{seq}(T, \text{seq}((C, \text{id}), S))}$$

[ALIGN-SLOT-SLOT]

$$\frac{\text{demandClass}(S\tau, S\rho) = \text{ordinary} \quad S; \Omega \vdash \tau \doteq \rho \dashv S'}{S; \Omega \vdash \tau \preceq \rho \dashv S'}$$

[ALIGN-ORDINARY]

product-of-matchers branch は whole-product matcher の生成と matcher-to-slot 消費を一つの checking cut で行う。non-ordinary class は resolved expected head が slot の場合だけであり、matcher-headed expected type は ordinary alignment へ退化する。

3.4 唯一の checking 規則

expected type は synthesis の入力にならず、synthesis が返した cut の S_1, Ω_1 でだけ alignment を選ぶ。

$$\frac{q; S; \Omega; \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \tau \preceq \rho \dashv S'}{q; S; \Omega; \Gamma \vdash e \Leftarrow \rho \dashv q_1; S'; \Omega_1}$$

[DD-CHECK]

4 式の synthesis 規則

α_{q_τ} は supply q が指す次の target variable である。list 判断は要素を左から右へ処理し、 q, S, Ω の三状態を順に渡す。instance の上付き ren は fresh capability binder image を renameOnly, str は structuralFlexible として ledger へ追加することを表す。

4.1 変数・関数・適用

変数 lookup は prevailing substitution を context に適用してから scheme を fresh instantiate する。lambda domain は fresh metavariable とする。application は function synthesis, function type との ordinary alignment, argument

checking の順で進む。

$$\begin{array}{c}
\frac{(S\Gamma)(x) = \sigma \quad \text{Inst}_q^{\text{ren}}(\Omega, \sigma) = (\tau, q', \Omega')}{q; S; \Omega; \Gamma \vdash x \Rightarrow \tau \dashv q'; S; \Omega'} \quad \frac{q + \langle 0, 1 \rangle; S; \Omega; \Gamma, x : \text{mono } \alpha_{q_\tau} \vdash e \Rightarrow \tau \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash \lambda x. e \Rightarrow \alpha_{q_\tau} \rightarrow \tau \dashv q'; S'; \Omega'} \\
\text{[DD-VAR]} \qquad \qquad \qquad \text{[DD-LAM]} \\
\\
\frac{q; S; \Omega; \Gamma \vdash e_1 \Rightarrow \tau_f \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \tau_f \doteq \alpha_{(q_1)_\tau} \rightarrow \alpha_{(q_1)_\tau+1} \dashv S_2 \quad q_1 + \langle 0, 2 \rangle; S_2; \Omega_1; \Gamma \vdash e_2 \Leftarrow \alpha_{(q_1)_\tau} \dashv q_2; S_3; \Omega_2}{q; S; \Omega; \Gamma \vdash e_1 e_2 \Rightarrow \alpha_{(q_1)_\tau+1} \dashv q_2; S_3; \Omega_2} \\
\text{[DD-APP]}
\end{array}$$

function domain が slot に解決された場合, argument の構文に関係なく, 最後の checking cut で matcher-to-slot coercion を選べる。

変数規則の instance は canonical scheme の finite opening から計算される。state erasure では, root の終端 substitution が冪等であることと opening transport を使い, この instance が終端 context の lookup と一致することを代数的に導く。variable node 自体に追加の terminal audit field はない。binder 名の masking や capture 条件も必要ない。

4.2 リテラル・タプル・コンストラクタ

$$\begin{array}{c}
\frac{}{q; S; \Omega; \Gamma \vdash n \Rightarrow \text{Int} \dashv q; S; \Omega} \quad \frac{q; S; \Omega; \Gamma \vdash \bar{e} \Rightarrow \bar{\tau} \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash (\bar{e}) \Rightarrow (\bar{\tau}) \dashv q'; S'; \Omega'} \\
\text{[DD-LIT]} \qquad \qquad \qquad \text{[DD-TUPLE]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma \vdash \bar{e} \Leftarrow \bar{\rho} \dashv q'; S'; \Omega_1 \quad \Omega' = \text{freezeExport}(S', \rho, \Omega_1)}{q; S; \Omega; \Gamma \vdash C \bar{e} \Rightarrow \rho \dashv q'; S'; \Omega'} \\
\text{[DD-CON]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_{\text{prim}}(op)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma \vdash \bar{e} \Leftarrow \bar{\rho} \dashv q'; S'; \Omega_1 \quad \Omega' = \text{freezeExport}(S', \rho, \Omega_1)}{q; S; \Omega; \Gamma \vdash op(\bar{e}) \Rightarrow \rho \dashv q'; S'; \Omega'} \\
\text{[DD-PRIM]}
\end{array}$$

freezeExport は exported result に残る structural leaf だけを rename-only にする。

4.3 Let と再帰

$$\begin{array}{c}
\frac{\sigma = \text{Gen}(\widehat{\Sigma}, S_1 \Gamma, S_1 \tau_1) \quad q; S; \Omega; \Gamma \vdash e_1 \Rightarrow \tau_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma, x : \sigma \vdash e_2 \Rightarrow \tau_2 \dashv q'; S'; \Omega' \quad S' \sigma = \text{Gen}(\widehat{\Sigma}, S' \Gamma, S' \tau_1)}{q; S; \Omega; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Rightarrow \tau_2 \dashv q'; S'; \Omega'} \\
\text{[DD-LET]} \\
\\
\frac{f \neq x \quad \text{DirectSelf}(f, e) \quad \text{NonMatcherBody}(e) \quad q + \langle 0, 2 \rangle; S; \Omega; \Gamma, f : \text{mono } (\alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1}), x : \text{mono } \alpha_{q_\tau} \vdash e \Rightarrow \rho \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \rho \doteq \alpha_{q_\tau+1} \dashv S'}{q; S; \Omega; \Gamma \vdash \text{fix } f \ x. e \Rightarrow \alpha_{q_\tau} \rightarrow \alpha_{q_\tau+1} \dashv q_1; S'; \Omega_1} \\
\text{[DD-FIX]}
\end{array}$$

let の局所安定性だけでは, 外側の後続 solve を越えた公開終端での一般化一致は表せない。そこで terminal audit は, root の終端 context と終端 value type から Gen を再計算し, body で使った scheme への終端作用と一致する事実を保持する。相互 erasure はこの事実を使って value-flow instance を再構成する。

$\text{DirectSelf}(f, e)$ は shadowing-aware な構文条件であり, f の自由出現を application spine の head に限る. matcher literal を body に持つ recursion は次章の専用 placeholder 規則を使う.

4.4 標準 matcher producer

$$\frac{}{q; S; \Omega; \Gamma \vdash \text{something} \Rightarrow \text{Matcher Any } \alpha_{q_\tau} \dashv q + \langle 0, 1 \rangle; S; \Omega} \quad [\text{DD-SOMETHING}]$$

something 自身は matcher producer である. slot が必要かどうかはこの規則では決めず, 外側の checking cut が expected head を見て決める.

5 Pattern と matcher の規則

5.1 ユーザーパターンの synthesis

ユーザーパターン判断を

$$q; S; \Omega; \Gamma; \Phi; \Delta \vdash p \Rightarrow \eta; \Delta' \dashv q'; S'; \Omega'$$

と書く. pattern dual $\eta = \kappa \triangleright \tau$ と binding context を同時に合成し, Δ を左から右へ延長する. fresh pattern capability は consumer の局所変数なので $\text{structuralFlexible}$ として ledger に追加する.

$$\frac{x \notin \text{dom}(\Delta) \quad \Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \$x \Rightarrow \chi_{q_\kappa} \triangleright \alpha_{q_\tau}; \Delta, x : \alpha_{q_\tau} \dashv q + \langle 1, 1 \rangle; S; \Omega'} \quad [\text{DD-PAT-VAR}]$$

$$\frac{\Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash _ \Rightarrow \chi_{q_\kappa} \triangleright \alpha_{q_\tau}; \Delta \dashv q + \langle 1, 1 \rangle; S; \Omega'} \quad [\text{DD-PAT-WILD}]$$

$$\frac{q; S; \Omega; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau \dashv q_1; S_1; \Omega_1 \quad \Omega' = \Omega_1[\chi_{(q_1)_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \#e \Rightarrow \chi_{(q_1)_\kappa} \triangleright \tau; \Delta \dashv q_1 + \langle 1, 0 \rangle; S_1; \Omega'} \quad [\text{DD-PAT-VALUE}]$$

$$\frac{\Phi(x) = \kappa \triangleright \tau}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \tilde{x} \Rightarrow \kappa \triangleright \tau; \Delta \dashv q; S; \Omega} \quad [\text{DD-PAT-EMBED}]$$

tuple と constructor は子を先に合成する. constructor scheme は structural instance を作り, field target alignment と capability projection の後, result に残る structural leaf を freeze する.

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}; \Delta' \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash (\bar{p}) \Rightarrow (\bar{\eta}.\bar{\kappa}) \triangleright (\bar{\eta}.\bar{\tau}); \Delta' \dashv q'; S'; \Omega'} \quad [\text{DD-PAT-TUPLE}]$$

$$\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_P(c)) = (\bar{p} \Rightarrow_P \rho, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}; \Delta' \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \bar{\eta}.\bar{\tau} \doteq \bar{p} \dashv S_2 \quad q_1; S_2; \Omega_1 \vdash_c \bar{\eta}.\bar{\kappa} \Rightarrow \kappa \dashv q_2; S_3; \Omega_2 \quad \text{CapCompatible}_{\widehat{\Sigma}}(c; S_3 \bar{\eta}.\bar{\kappa}, S_3 \kappa) \quad \Omega' = \text{freezeExport}(S_3, \kappa \triangleright \rho, \Omega_2)}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash c \bar{p} \Rightarrow \kappa \triangleright \rho; \Delta' \dashv q_2; S_3; \Omega'} \quad [\text{DD-PAT-CON}]$$

ここで $\vdash_c$ は signature entry が定める constructor capability projection である。規則中の **CapCompatible** はこの局所 cut での検査である。公開終端まで後続 solve が続く場合に備え、terminal audit は終端 substitution を dual 列と result capability へ適用した後の **CapCompatible** を再検査する。state erasure はこの終端事実を使うため、局所 capability が以後不変であることを仮定しない。

$p_1 \& p_2$ は左の bindings を右へ渡す。 $p_1 \mid p_2$ は同じ入力 Δ から両 alternative を調べ、dual と binder 名・型を最後に align する。

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \Rightarrow \eta_1; \Delta_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \Phi; \Delta_1 \vdash p_2 \Rightarrow \eta_2; \Delta_2 \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \eta_1 \doteq \eta_2 \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \& p_2 \Rightarrow \eta_1; \Delta_2 \dashv q_2; S'; \Omega_2}$$

[DD-PAT-AND]

$$\frac{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \Rightarrow \eta_1; \Delta_1 \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \Phi; \Delta \vdash p_2 \Rightarrow \eta_2; \Delta_2 \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \eta_1 \doteq \eta_2 \dashv S_3 \quad S_3; \Omega_2 \vdash \Delta_1 \doteq \Delta_2 \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash p_1 \mid p_2 \Rightarrow \eta_1; \Delta_1 \dashv q_2; S'; \Omega_2}$$

[DD-PAT-OR]

pattern-function scheme の capability binder は instance 時点から rename-only である。

$$\frac{\text{InstDual}_q^{\text{ren}}(\Omega, \widehat{\Sigma}_F(f)) = (\bar{\eta} \Rightarrow \eta, q_0, \Omega_0) \quad q_0; S; \Omega_0; \Gamma; \Phi; \Delta \vdash \bar{p} \Rightarrow \bar{\eta}'; \Delta' \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \bar{\eta}' \doteq \bar{\eta} \dashv S'}{q; S; \Omega; \Gamma; \Phi; \Delta \vdash f \bar{p} \Rightarrow \eta; \Delta' \dashv q_1; S'; \Omega_1}$$

[DD-PAT-APP]

5.2 MatchAll の型付け

matchAll は target を合成し、pattern target と align した後、matcher expression を slot に対して check する。第四 premise はこの規則が直接導入する唯一の slot demand である。matcher expression が matcher literal なら、その内部の各 next matcher も別の checking cut を持つ。

$$\frac{q; S; \Omega; \Gamma \vdash e_t \Rightarrow \tau_t \dashv q_1; S_1; \Omega_1 \quad q_1; S_1; \Omega_1; \Gamma; \emptyset; \emptyset \vdash p \Rightarrow \kappa_p \triangleright \tau_p; \Delta \dashv q_2; S_2; \Omega_2 \quad S_2; \Omega_2 \vdash \tau_p \doteq \tau_t \dashv S_3 \quad q_2; S_3; \Omega_2; \Gamma \vdash e_m \Leftarrow \text{MatcherSlot } \kappa_p \tau_t \dashv q_3; S_4; \Omega_3 \quad q_3; S_4; \Omega_3; \text{mono } \Delta, \Gamma \vdash e \Rightarrow \tau_r \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash \text{matchAll } e_t \text{ as } e_m \text{ with } p \rightarrow e \Rightarrow \text{List } \tau_r \dashv q'; S'; \Omega'}$$

[DD-MATCHALL]

5.3 Primitive pattern と data pattern

primitive pattern 判断は shared matcher target に対して hole dual 列と bindings を返す. hole だけが fresh consumer capability を生成する.

$$\begin{array}{c}
\frac{\Omega' = \Omega[\chi_{q_\kappa} \mapsto \text{structuralFlexible}]}{q; S; \Omega \vdash \$ \Leftarrow \tau \Rightarrow [\chi_{q_\kappa} \triangleright \tau]; \emptyset \dashv q + \langle 1, 0 \rangle; S; \Omega'} \quad \frac{}{q; S; \Omega \vdash _ \Leftarrow \tau \Rightarrow []; \emptyset \dashv q; S; \Omega} \\
\text{[DD-PP-HOLE]} \quad \text{[DD-PP-WILD]} \\
\\
\frac{}{q; S; \Omega \vdash \# \$ x \Leftarrow \tau \Rightarrow []; x : \tau \dashv q; S; \Omega} \\
\text{[DD-PP-VALUE]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_P(c)) = (\bar{\rho} \Rightarrow_P \rho, q_0, \Omega_0) \quad S; \Omega_0 \vdash \rho \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega_0 \vdash \overline{pp} \Leftarrow \bar{\rho} \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega_1}{q; S; \Omega \vdash c \overline{pp} \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega_1} \\
\text{[DD-PP-CON]} \\
\\
\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad S; \Omega \vdash (\bar{\alpha}) \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega \vdash \overline{pp} \Leftarrow \bar{\alpha} \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash (\overline{pp}) \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta \dashv q'; S'; \Omega'} \\
\text{[DD-PP-TUPLE]}
\end{array}$$

data pattern 判断は arm の bindings を返す.

$$\begin{array}{c}
\frac{}{q; S; \Omega \vdash \$ x \Leftarrow \tau \Rightarrow x : \tau \dashv q; S; \Omega} \quad \frac{}{q; S; \Omega \vdash _ \Leftarrow \tau \Rightarrow \emptyset \dashv q; S; \Omega} \\
\text{[DD-DP-VAR]} \quad \text{[DD-DP-WILD]} \\
\\
\frac{\text{Inst}_q^{\text{str}}(\Omega, \widehat{\Sigma}_D(C)) = (\bar{\rho} \rightarrow \rho, q_0, \Omega_0) \quad S; \Omega_0 \vdash \rho \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega_0 \vdash \overline{dp} \Leftarrow \bar{\rho} \Rightarrow \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash C \overline{dp} \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'} \\
\text{[DD-DP-CON]} \\
\\
\frac{\text{freshTargets}(n, q) = (\bar{\alpha}, q_0) \quad S; \Omega \vdash (\bar{\alpha}) \doteq \tau \dashv S_1 \quad q_0; S_1; \Omega \vdash \overline{dp} \Leftarrow \bar{\alpha} \Rightarrow \Delta \dashv q'; S'; \Omega'}{q; S; \Omega \vdash (\overline{dp}) \Leftarrow \tau \Rightarrow \Delta \dashv q'; S'; \Omega'} \\
\text{[DD-DP-TUPLE]}
\end{array}$$

5.4 Clause と matcher literal

一つの clause は primitive pattern, next-matcher 列, arm 列の順に処理する. 各 next matcher は対応する hole slot に対して同じ checking 判断を使う.

$$\frac{q; S; \Omega \vdash pp \Leftarrow \tau \Rightarrow \bar{\eta}; \Delta_{pp} \dashv q_1; S_1; \Omega_1 \quad \text{decomposeME}(M, |\bar{\eta}|) = \text{some } \bar{M} \quad q_1; S_1; \Omega_1; \Gamma \vdash \bar{M} \Leftarrow \text{MatcherSlot } \eta.\kappa.\eta.\tau \dashv q_2; S_2; \Omega_2 \quad q_2; S_2; \Omega_2; \Gamma; \Delta_{pp} \vdash \bar{a} \Leftarrow \tau; \text{List prod}(\bar{\eta}.\bar{\tau}) \dashv q'; S'; \Omega'}{q; S; \Omega; \Gamma \vdash pp \text{ as } M \text{ with } \bar{a} \Leftarrow \tau \Rightarrow \bar{\eta} \dashv q'; S'; \Omega'} \\
\text{[DD-CLAUSE]}$$

matcher literal は全 clause で一つの fresh target を共有する. 終端 substitution で hole dual を解決して evidence を再計算し, shape, catch-all order, binder 線形性, arm exhaustiveness, coverage を検査する. 最後に, final capability

に現れる全 DD-owned explicit ledger key の structural leaf だけを freeze する。

$$\frac{\begin{array}{c} q + \langle 0, 1 \rangle; S; \Omega; \Gamma \vdash cls \Leftarrow \alpha_{q_r} \Rightarrow \bar{\eta} \dashv q'; S'; \Omega_1 \\ \text{collectClauseEvidence}_{\bar{\Sigma}}(cls, \text{terminalHoleCaps}(S', \bar{\eta})) = \text{some } \bar{e}v \quad \text{inferShape}_{\bar{\Sigma}}(\bar{e}v) = \text{some } \kappa \\ \text{clauseCapsListCheck}_{\bar{\Sigma}}(\kappa, cls, \text{terminalHoleCaps}(S', \bar{\eta})) \quad \text{CatchAllLast}(cls) \quad \text{MatcherBindersCheck}(cls) \\ \text{ArmExhaustive}_{\bar{\Sigma}_D}(cls, S' \alpha_{q_r}) \quad \text{CoverageOK}_{\bar{\Sigma}}(cls, \kappa) \quad \Omega' = \text{freezeMatcher}(S', \kappa, \Omega_1) \end{array}}{q; S; \Omega; \Gamma \vdash \text{matcher } cls \Rightarrow \text{Matcher } \kappa \alpha_{q_r} \dashv q'; S'; \Omega'}$$

[DD-MATCHER]

上の evidence は matcher 自身の局所終端 S' で計算される。matcher が外側の式の子である場合、root 終端までの rename-only suffix で capability 名が変わり得る。terminal audit は root 終端で hole capability を解決し直し、evidence 収集, shape, clause capability, arm exhaustiveness, coverage を再検査する。matcher の相互 erasure はこの正規化済み事実から `RuntimeTyping` constructor を作り、局所の solver trace や MGU を runtime certificate へ残さない。

matcher literal を body に持つ recursion は、clause skeleton から placeholder を作る専用規則を使う。placeholder range の capability key は matcher 開始前に structural-flexible として ledger へ追加される。

$$\frac{\begin{array}{c} f \neq x \quad \text{DirectSelf}(f, \text{matcher } cls) \\ \text{fixMatcherPlaceholderSupply}_{\bar{\Sigma}}(cls, q) = \text{some } (\tau_d, \tau_c, q_0) \quad \Omega_0 = \text{markCapRange}(q, q_0, \Omega) \\ q_0; S; \Omega_0; \Gamma, f : \text{mono } (\tau_d \rightarrow \tau_c), x : \text{mono } \tau_d \vdash \text{matcher } cls \Rightarrow \rho \dashv q_1; S_1; \Omega_1 \quad S_1; \Omega_1 \vdash \rho \doteq \tau_c \dashv S' \end{array}}{q; S; \Omega; \Gamma \vdash \text{fix } f \ x. (\text{matcher } cls) \Rightarrow \tau_d \rightarrow \tau_c \dashv q_1; S'; \Omega_1}$$

[DD-FIX-MATCHER]

`NonMatcherBody` により通常の DD-FIX と排他的である。

6 機械化された性質と接続

本節では、前節までの規則から得られる性質と、実行可能推論・動的安全性への接続をまとめる。solver の個別補題, trace invariant, 回帰一覧は `../docs/details.md` に分離する。

6.1 DDTyping 自身の性質

Slot demand

$S; \Omega \vdash \tau \preceq \rho \dashv S'$ が ordinary equality でないなら, resolved expected type は必ず slot-headed である。

$$S; \Omega \vdash \tau \preceq \rho \dashv S' \implies (S; \Omega \vdash \tau \doteq \rho \dashv S') \vee \exists \kappa, v. S\rho = \text{MatcherSlot } \kappa v.$$

特に $S\rho$ が matcher-headed なら ordinary alignment に限られる。これにより「coercion は slot demand を観測した checking cut で一度だけ」が judgment の性質になる。

三状態の extension

すべての raw DD family は supply を単調に進め, 出力 substitution を入力 substitution と局所 solve delta の chronological replay に因子化できる。対応する Origin family は raw derivation と同じ構造を持ち, ledger transition が入力 ledger を保ち, fresh supply の範囲内だけを追加・freeze することを保証する。公開型, pattern dual, bindings, hole ledger に現れる flexible variable は終端 supply で有界である。

Solved form

capability-only, target-only, paired, 二段階 matcher / slot solve, one-way matching の各局所操作は, 入力 substitution の冪等性を保存する。これを traversal 順に合成し, alignment family と, expression, expression list, checking,

user / primitive / data pattern, pattern list, arm, clause, pattern-constructor capability を含む全 14 raw DD family について

$$S.\text{Idempotent} \implies S'.\text{Idempotent}$$

を証明済みである。公開判断は恒等 substitution から始まるため、root の終端 substitution は無条件に solved form である。

No guess と freeze

exact MGU は constraint 外で恒等, constraint range 内, solved form である。さらに Origin admissibility により rigid key を固定し, rename-only key の structural strengthening を拒否する。constructor / fresh consumer の structural variable は局所 solve を許すが, 選択的 export と matcher finalization を越えた producer leaf は rename-only になる。let は終端 substitution 下の generalization stability も要求する。

帰りの現在地

origin を追跡しない局所導出では, 量化 producer capability を後続 solve が Any へ強化する *capFreezeProgram* と, let-generalized capability を強化する *letCapFreezeProgram* の問題が現れる。これらは現行 public DDTyping の正例ではない。該当する局所 delta が Origin-aware solve で拒否されることは証明済みであり, プログラム全体についても $\neg \text{DDTyping}(\text{capFreezeProgram})$ と $\neg \text{DDTyping}(\text{letCapFreezeProgram})$ を閉じる negative regression を構成済みである。or-pattern, delegating matcher, let-polymorphic producer の public positive Origin certificate も構成済みである。

6.2 実行可能推論から DDTyping へ

公開 inference は停止する raw traversal と有限の fail-closed validator の合成である。

$$\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{bind}(\text{inferRaw}(\widehat{\Sigma}, \Gamma, e), \lambda r. \text{if } \text{wBridgeCheck}(\widehat{\Sigma}, r) \text{ then some } r \text{ else none}).$$

raw traversal の executable ledger と DD の intrinsic Origin certificate は同じ policy を別の役割で記録する。前者は solver を実行時に fail closed にし, 後者は関係的 derivation の solve admissibility を証明する。validator は trace から binder-local instance, alignment, matcher finalization, let generalization, freeze event を再検査する。成功した fuelled traversal は, expression synthesis / checking, expression list, pattern, matcher, arm, clause を含む 10 family の相互帰納により, 同じ supply, prevailing substitution, origin ledger を持つ raw DD derivation へ再構成される。append-only history は各再帰呼出しを一つの root 終端へ接続し, validator の証拠から let, matcher, pattern constructor の terminal audit を構成する。従って公開成功から次を得る。

$$\frac{\text{infer}(\widehat{\Sigma}, \Gamma, e) = \text{some } r}{\text{DDTyping}(\widehat{\Sigma}, \Gamma, e, r.\text{resolvedTarget})}.$$

この定理は RuntimeTyping を経由せず, caller-supplied bridge, history, InferenceInputWF を前提にしない。

これとは独立に, $S_r = r.\text{state}.\text{prevailing}$ として公開成功から $\text{RuntimeTyping}(\widehat{\Sigma}, S_r, \Gamma, e, r.\text{resolvedTarget})$ を再構成する既存経路もある。RuntimeTyping は source acceptance ではなく, solver state を持たない内部 certificate であり, 動的メタ理論との直接接続に用いる。一般 context について二つの経路を同一視する主張はしない。

6.3 動的安全性

freeze 済み signature の checker が FrozenSigWF を確立する。その下で, typed environment における評価は RuntimeTyping を ValueTy へ保存する。matching machine について, typed state の一段保存, 局所 progress, 到達可能 state の保存, 成功 branch の substitution typing を証明済みである。空 successor 列は正当な match failure であり stuck ではない。一般の program termination は主張しない。

6.4 DDTyping から安全性への構成要素

目標とする source-facing な公開安全性定理の構造は模式的に次である．ここで `RuntimeSafe` は Lean の単一 judgment 名ではなく、`CoreSafety` が束ねる `preservation`, `progress`, 到達可能性, 成功 branch typing を略記する．

$$\widehat{\Sigma}.\text{SchemesClosed} \implies \text{DDTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{RuntimeTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{RuntimeSafe}(\widehat{\Sigma}, e, \tau).$$

第一の矢印と、`RuntimeTyping` を前提とする `CoreSafety` の各構成要素は機械化済みである．`DDTyping` を入口として後者を一つに束ねる公開定理は今後の接続である．第一の矢印では、入力 cut より前の origin policy を保存する supply-scoped な残余 substitution と、各局所出力を一つの root 終端 substitution へ結ぶ `StateFactorization` を使う．全 14 Origin family の factorization と constructor-wise erasure を一つの終端 cut へ固定した相互帰納にまとめ、supply, prevailing substitution, ledger を消去する．

局所 cut ではなく公開終端で再確認すべき追加情報は terminal audit に分離する．その facts は三種であり、`let` は終端 context / value type から再計算した generalization の一致、matcher は terminal hole capability から再計算した evidence, shape, clause capability, arm exhaustiveness, coverage, pattern constructor は終端 dual / capability の `CapCompatible` を記録する．variable node は追加 facts を持たず、canonical Scheme の finite opening transport から終端 lookup との一致を直接導く．相互 erasure はこれらを使って `RuntimeTyping`, pattern, arm, clause certificate を再構成する．matcher-to-slot は終端 `CapabilityDemand`, slot-to-slot は終端型等式へ消去される．

従って closed signature 上の closed-program について

$$\widehat{\Sigma}.\text{SchemesClosed} \implies \text{DDTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{RuntimeTyping}(\widehat{\Sigma}, \emptyset, e, \tau)$$

が成立する．certificate の存在を DD rule の premise に置く循環的な定義は用いず、DD derivation 自身の solved-form 保存, Origin history, terminal audit から射影する．

6.5 受理完全性

もう一つの未完成定理は、DD derivation を executable inference の成功へ移すことである．executable selector と DD rule の cut-resolved classification を一致させ、exact solve witness と Origin transition を executable trace および terminal validator に対応付ける必要がある．目標は追加 premise のない

$$\text{DDTyping}(\widehat{\Sigma}, \emptyset, e, \tau) \implies \text{infer}(\widehat{\Sigma}, \emptyset, e) \neq \text{none}$$

である．

6.6 機械化の範囲

全 expression / pattern / arm / clause の raw DD family, それらに対応する intrinsic Origin family, ledger の基本 transition, alignment の slot-demand, state replay, supply boundedness, 全 14 raw family の solved-form 保存, terminal audit, terminal-fixed な相互 runtime erasure, closed-program の `DDTyping` $\implies$ `RuntimeTyping`, 公開 inference から `DDTyping` および `RuntimeTyping` への reconstruction, concrete dynamic safety は Lean 4 に機械化済みである．public freeze 回帰も閉じている．受理接続に関しては、逆向きの DD / infer 受理完全性だけが未完成である．source-facing な公開安全性の bundled theorem は別の今後の接続である．主定理と一般補題に `sorry`, `admit`, project-defined axiom はない．